CeTTa-Hosted Guarded Supercompilation for MeTTa: CettaPacks, Multi-Geometry IR, and Proof-Aware Mixed Execution

Ben Goertzel + various AI tools

July 6, 2026

Abstract

This note proposes a CeTTa-hosted architecture for aggressive MeTTa supercompilation into mixed artifacts: generated C kernels, PathMap/MORK indices and ACT artifacts, MM2 fragments, tensor or factor kernels, and residual MeTTa fallbacks. The core claim is that the best system is not "compile all MeTTa to MM2", not "let PeTTa handle all search", and not "do all supercompilation and execution inside MeTTaIL". The stronger architecture is a guarded mixed-execution system. CeTTa hosts the runtime artifacts and exact fallback path; LeaTTa and Lean provide a semantic and certificate layer; a multi-geometry IR delays commitment among C, PathMap/trie, factor, tensor, MM2, and residual MeTTa presentations; and CettaPacks carry explicit dependency, effect, structural, profile, and space guards.

The body of the document is written for humans: concepts, motivations, architectural decisions, and process. The appendix is written for coding agents: concrete interfaces, manifest fields, pseudocode, tests, and anti-patterns. The entire source is ASCII-only LaTeX.

Executive Summary

The thesis

The recommended architecture is:

> Use LeaTTa and Lean to check what transformations mean. Use a multi-geometry IR to decide what operational shape a hot computation should take. Use CeTTa to host the resulting C kernels, PathMap/MORK artifacts, MM2 fragments, tensor or factor kernels, and residual MeTTa fallbacks under explicit runtime guards.

This is a proposal for *guarded mixed execution*, not for a single new universal backend. The supercompiler should produce CettaPacks: loadable artifacts containing generated code, index artifacts, residual programs, certificates, and manifests. A CettaPack is valid only when all of its assumptions still hold. If the assumptions hold, CeTTa calls the specialized kernel. If they are slightly stale but patchable, CeTTa updates the artifact or its indices. If the assumptions have drifted too far, CeTTa replans or falls back to generic evaluation. The central safety property is that failed optimization assumptions degrade to ordinary CeTTa behavior rather than corrupting results.

The architecture begins with a conservative observation: MeTTa optimization is not one problem. Hot regions in MeTTa programs differ radically. Some are exact pattern queries over a large mostly static space. Some are branch-heavy proof searches. Some are monotone forward saturations. Some are PLN computations better expressed as factor-graph demand and supply. Some are prefix-local mining or weighted trie operations. Some are tensorizable semiring contractions. Some are reflective or dynamic code regions that should remain residual MeTTa. A supercompiler that lowers everything too early to raw MM2, or too early to a single internal language, throws away the very structure that would let it choose well.

The architecture therefore uses a multi-geometry intermediate representation, or MG-IR. The MG-IR separates the semantic object from its operational presentation. The same source region may be presented as native C control, a PathMap/trie query, a factor graph, a tensor program, an MM2 monotone rule set, or residual MeTTa. The planner chooses among those presentations using profile data, effect analysis, space schemas, cache validity, exactness requirements, and cost models. Humans define the semantic equivalences, laws, and safe lowerings. The supercompiler specializes within that declared space.

Why CeTTa is the right host

CeTTa adds value because it sits at the right boundary between MeTTa semantics and low-level execution. It is a direct C runtime rather than only a compiler pipeline. It exposes a named profile family, including HE-compatibility and extensions. It has structured spaces, snapshot surfaces, PathMap/MORK integration, and compiled ACT-style artifact loading. It can call native exact matching after indexed candidate selection. It can load generated C and use the same symbol IDs, arenas, visitors, and fallback evaluator as the ordinary runtime. These are not incidental implementation details; they are precisely what a guarded supercompiler needs.

The simplest high-impact optimization illustrates the point. For a hot MeTTa expression such as:

```
(match &kb (target rs10000544 $gene) $gene)
```

or a larger conjunction over several spaces, a CettaPack should not replace the entire evaluator with PathMap. Instead it should generate a site-specific kernel:

```
1. Check the CettaPack guard vector.
2. Use PathMap or MORK to narrow candidates.
3. Use a product zipper or trie join when several patterns interact.
4. Call CeTTa native seeded rematch for exact binding semantics.
5. Construct or stream result atoms directly in C.
6. Preserve bag multiplicity from the source space records.
7. Fall back to generic CeTTa if any assumption fails.
```

This pattern repairs a major weakness of shallow PathMap integration without taking the dangerous step of turning trie keys into the meaning of MeTTa matching. PathMap narrows; native rematch decides. PathMap may store canonical support; multiplicity remains separate. Co-reference, binder hygiene, variable equality, and result construction remain the responsibility of the exact matcher and the specialized result builder.

CeTTa also adds value in search-heavy regions, even though PeTTa is currently stronger on some pure branching workloads. The right lesson from PeTTa is not to abandon CeTTa; it is to add a local search machine inside the CeTTa-hosted artifact system. Backward chaining can be compiled into a local episode with choice frames, trail/watermark rollback, scratch arenas, fixed rule signatures, PathMap-backed fact iterators, exact boundary rematch, and a streaming answer visitor. PeTTa may also be used as an intermediate backend for selected pure search fragments, but CeTTa remains the host that guards the artifact, manages spaces, checks profile assumptions, loads indices, and falls back.

Greg Meredith's inevitable question: why not do all the supercompilation and execution in MeTTaIL?

The short answer is that MeTTaIL is an excellent contract and compilation layer, but the proposed system needs a runtime artifact host that lives next to mutable spaces, PathMap/MORK indices, ACT artifacts, MM2 stepping, generated C kernels, exact native rematch, profile gates, and deoptimization paths. In this design, MeTTaIL can be a valuable front-end, contract language, proof interface, or source of verified compilation recipes. It should not be forced to become the entire runtime substrate for every specialized execution product.

The longer answer has five parts.

First, the most valuable optimizations depend on runtime resources, not only on source syntax. A hot query kernel may depend on the schema of three spaces, their revision or epoch vector, their current PathMap coverage, their selectivity statistics, their multiplicity records, and the profile under which they are interpreted. Those are runtime conditions. A static compilation product may describe them, but the host must check them at call time. CeTTa is already the runtime where those spaces, profiles, and backends live.

Second, the target is mixed. A real artifact may contain generated C for control and arithmetic, PathMap or ACT files for indexed access, MM2 for sink-free monotone saturation, a residual MeTTa fallback for reflective or unsupported code, and perhaps a PeTTa delegate for a local pure search episode. Calling this entire bundle "MeTTaIL execution" would either be inaccurate or would require MeTTaIL to reimplement the same mixed artifact host. The proposal is cleaner: let MeTTaIL describe or certify parts of the transformation, and let CeTTa host the resulting mixed runtime artifact.

Third, exactness in the first killer pass depends on CeTTa's native matcher. The safe PathMap strategy is not to trust an index as a matcher. It is to use PathMap for candidate narrowing and then call CeTTa's native seeded rematch as the source of exact binding truth. If one routes everything through an IL-only execution path, one must either recreate that exact runtime boundary or accept a riskier matching story. The CeTTa-hosted design keeps the trusted seam small and explicit.

Fourth, supercompiled artifacts need deoptimization and incremental maintenance. A space may grow append-only by five percent. A profile may remain unchanged but selectivity statistics may drift. A meta-rule may alter the normalized shape of a match site. A language definition pack may be upgraded. A schema may migrate under the same human-readable name. The CettaPack lifecycle therefore has several states: valid, patchable, replan, and invalid. That lifecycle belongs in the execution host. MeTTaIL can describe assumptions, but CeTTa is where those assumptions are checked against live resources.

Fifth, the performance win is partly about being close to C and MORK. For large-KB workloads, the cost of atom construction, arena use, streaming answer visitors, candidate iteration, rematch, and multiplicity handling matters. A generated C kernel inside CeTTa can use symbol IDs, arenas, native space records, MORK handles, direct PathMap iterators, and visitor callbacks without marshaling through a higher-level interpreter boundary. This is where CeTTa's low-level nature pays off: the fast path can be a compact guard check followed by in-process C loops over resident data structures.

Trying to replicate this role inside MeTTaIL would leave two unattractive choices. One could run the optimized region through an IL interpreter or higher-level execution layer, paying avoidable allocation, dispatch, and marshaling costs exactly where the kernel is supposed to be fastest. Or one could bolt a native C/MORK/PathMap runtime, guard table, artifact loader, and exact rematch boundary onto MeTTaIL. But that second choice is essentially a second CeTTa-like host, with extra integration surface and greater risk of semantic drift. MeTTaIL can help prove, normalize, or construct such kernels, but forcing all execution through MeTTaIL would risk losing the locality and low-level control that motivate CeTTa.

Thus the answer is not "do not use MeTTaIL". The answer is: use MeTTaIL where it is strongest, as a contract-first and proof-aware compilation layer. Use CeTTa where it is strongest, as the guarded host for live mixed artifacts. The proposed boundary is complementary, not competitive.

Guarded identity must include code shape

A CettaPack cannot be treated merely as a kernel guarded by a profile, a space schema, and a space revision. MeTTa programs can evolve, and MeTTa has meta-level rules that can rewrite rules or change the effective structure of a match site. If a meta-rule changes the normalized query pattern but the data space does not change, a space revision guard alone will not notice. This is a correctness issue, not a mere engineering refinement.

A CettaPack must therefore be guarded by a dependency vector, not by a single space revision. Its identity should include:

```
source span hash
macro-expanded or meta-expanded region hash
match pattern structural hash
operator schema hashes
language definition pack digest
profile digest
effect policy digest
space dependency vector
artifact dependency vector
statistics signature
approximation or exactness signature
```

The structural hash is essential. It answers the question: is this still the same semantic region that the kernel was generated for? A kernel that was compiled for one normalized match pattern must not be reused for a different pattern merely because the same named source site still exists. Meta-rule-driven program evolution makes code shape part of the runtime dependency graph.

This point also strengthens the case for CeTTa. CeTTa already treats profiles, rule packs, extension surfaces, and runtime semantics as named objects. A CettaPack can include the digest of the relevant language definition pack and rule schema set. A host that treats semantics as an implicit property of whatever code happens to run is a worse fit for this kind of invalidation discipline.

Cross-space guards and incremental maintenance

Real MeTTa programs often query multiple spaces at once. A generated kernel for a cross-space join may depend on an evidence space, a rule space, a derived-fact space, and a statistics sidecar. The manifest must therefore hold a vector of per-space dependencies:

```
space id
role in the query plan
schema hash
revision or epoch policy
isolation mode
statistics signature
multiplicity source
PathMap coverage or artifact hash
```

A guard passes only when all code, profile, effect, schema, space, statistics, and artifact guards pass. This does not mean every minor change forces full fallback. The guard lifecycle should be richer:

State	Meaning
Valid	All guards pass. Run the kernel as-is.
Patchable	The change is append-only, bounded, and compatible with the kernel. Patch PathMap entries, statistics, or delta tables, then run.
Replan	The region is still meaningful but cost assumptions or join orders are stale. Use generic CeTTa now and regenerate asynchronously.
Invalid	Semantic assumptions failed. Fall back immediately.

This improves both performance and correctness. Append-only growth of a large space should not throw away a specialized kernel every time a few atoms are added. But a schema migration, a profile change, a destructive write, a meta-rule change, or a structural hash mismatch must invalidate or replan. The manifest should therefore include an amortization window: compile cost, expected calls, expected validity duration, maximum patch cost, and maximum allowed statistics drift. A region should be supercompiled only if it is hot and stable enough to pay back the specialization cost.

Effects are not binary

The effect field cannot be just “ReadOnly” versus “Mutation”. The supercompiler needs a richer vocabulary:

```
Read(resource, scope)
```

```
Append(resource, scope)
Write(resource, scope)
Observe(resource)
Create(resource)
Delete(resource)
```

These effects interact with isolation modes such as:

```
SnapshotIsolated
AppendOnlyEpoch
LockedRead
MutationBarrier
Immutable
```

This distinction matters. A read over an immutable space is different from a read over a snapshot, which is different from a read over a live append-only epoch, which is different from a read that races destructive writes. MM2 lowering is safe primarily for monotone append-only and sink-free regions. Native C kernels may be safe under snapshots or locks. Destructive shared mutations usually need barriers or residual MeTTa fallback.

LeaTTa and Lean should be phased in carefully

The proof story should be ambitious but not block the first useful kernel. Milestone 1 should emit certificates as structured data and rely on differential tests against generic CeTTa, HE where appropriate, and PeTTa where appropriate. Milestone 2 should Lean-check selected transformation classes. Milestone 3 should require proof-carrying admission for trusted kernel families.

The first proof target should be native seeded rematch parity, because the whole PathMap strategy depends on it. The essential theorem schema is:

> If PathMap candidate selection is sound, and every candidate is passed to CeTTa native seeded rematch, then the generated query kernel returns the same result bag as direct matching on the accepted fragment.

This keeps the trusted base small. The supercompiler does not initially need to prove arbitrary generated C correct. It needs to show that a kernel is an instance of a checked backend schema, and that the source region was transformed into that schema by checked or checkable steps.

What should be built first

The first implementation target should be a hot match kernel over a read-only or append-only space. It is high value, narrow enough to test, and central to later work. The kernel should use PathMap only for candidate selection and CeTTa native rematch for exactness. It should preserve multiplicity, test co-reference, refuse unsupported patterns, and have manifest-guarded fallback.

The second target should be the CettaPack loader and guard lifecycle: valid, patchable, replan, invalid. Cross-space dependency vectors and structural hashes should be present from the beginning, even if some fields are conservative placeholders at first.

The third target should be certificate emission and differential benchmarking. Benchmarks must include generic CeTTa, compiled CeTTa, PeTTa when applicable, HE when applicable, and guard overhead. A kernel that wins only when guard checks are excluded is not production-ready.

The fourth target should be either PeTTa-interoperable pure search episodes or a native local search episode. The native episode is more ambitious: choice frames, trail/watermark rollback, scratch arenas, and streamed answers. PeTTa interop may be a useful bridge, provided the contract is explicit.

PLN factor kernels, monotone MM2 lowerings, and tensor/GPU paths should come later. They should not be forgotten, but they depend on getting match, guards, effects, and artifact packaging right.

The design slogan

The architecture is:

> CeTTa-hosted supercompilation should produce not just optimized code, but guarded, incrementally maintainable, provenance-carrying CettaPacks whose identity includes both the code shape and the spaces they depend on.

That slogan also explains the division of labor across the ecosystem. PeTTa remains the search baseline and possible delegated backend. MeTTaIL remains a contract-first compilation and formalization layer. LeaTTa and Lean remain the semantic and certificate layer. MORK and MM2 remain the high-performance substrate for indexed and monotone execution. CeTTa is the host where all of these can be guarded, loaded, patched, deoptimized, and run.

Contents

Executive Summary **1**

1 Motivation and Scope **9**

2 Source Landscape **9**

3 Architecture at a Glance **9**

4 Why CeTTa Adds Value **10**

4.1 CeTTa is a host, not just an implementation language 10

4.2 CeTTa has explicit semantic profiles 11

4.3 CeTTa has the conservative PathMap boundary 11

4.4 CeTTa exposes mutation and snapshot surfaces 11

5 Why Not Just PeTTa? **12**

6 Why Not Just MeTTaIL? **12**

6.1 The live resources matter 12

6.2 Mixed artifacts are not a single IL 13

6.3 Replicating CeTTa inside MeTTaIL would add a costly layer 13

6.4 Exact rematch is a runtime seam 14

6.5 Deoptimization and patching are host responsibilities 14

6.6 The recommended relationship 14

7 The Multi-Geometry IR **15**

8 CettaPack v0.2 **15**

8.1 Artifact shape 15

8.2 Dependency vector 16

8.3 Meta-rule invalidation 17

8.4 Guard states 17

9 Effect Algebra and Isolation **17**

10 Pattern 1: Hot Match to PathMap plus Native Rematch **18**

10.1 Multiplicity protocol 19

10.2 Correctness contract 19

11 Pattern 2: Search Episodes, MAM, and PeTTa Interop **19**

12 Pattern 3: PLN Factor Kernels **20**

13 Pattern 4: Monotone MM2 Lowering **21**

14 Structural Hashes and Language Definition Packs **21**

15 Incremental Maintenance and Temporal Stability **22**

16 Certificates and Verification Phasing **22**

16.1 Phase 1: data certificates and differential testing 23

16.2 Phase 2: check selected transformations 23

16.3 Phase 3: proof-carrying CettaPack admission 23

17 Benchmarking Discipline **23**

18 Hyperseed and OmegaHive Framing **24**

19 Tensor and GPU Roadmap **25**

20 Roadmap **25**

21 Conclusion **27**

A Appendix for Coding Agents **27**

A.1 Prime directive 27

A.2 CettaPack manifest v0.2 sketch 27

A.3 Guard check pseudocode 29

A.4 Patch lifecycle pseudocode 30

A.5 Structural hash computation 30

A.6 MVP match kernel interface 31

A.7 MVP match kernel algorithm 31

A.8 Certificate skeleton 32

A.9 Differential tests for match kernels 32

A.10 Anti-patterns 33

A.11 Definition of done for milestone 1 34

1 Motivation and Scope

MeTTa is attractive because it gives humans a high-level language for symbolic, probabilistic, reflective, and meta-level computation. The problem is that many serious AI workloads ask for both high-level expression and low-level throughput. A PLN query over a genomic KB, a multi-pattern join over a PathMap-indexed space, a typed backward chainer over millions of candidate proof paths, or a repeated fitness kernel in a learning loop can spend most of its time in a small number of hot semantic regions.

Supercompilation is the natural idea: specialize a program to what is known about its workload, background spaces, profiles, and query patterns. But a naive supercompiler has two failure modes. If it lowers too early to raw MM2, it may hide the very structure needed for good planning. If it tries to generate a full replacement evaluator, it risks changing semantics around matching, multiplicity, nondeterminism, mutation, and reflection. The architecture here avoids both mistakes.

The proposal is deliberately selective. It does not ask the system to supercompile every expression. It asks the runtime to identify hot, stable, semantically classifiable regions and replace them with guarded artifacts. The generic CeTTa evaluator remains the fallback and the semantic reference for dynamic or unsupported cases.

2 Source Landscape

This proposal synthesizes four families of ideas.

First, CeTTa provides a direct C runtime for MeTTa with explicit profiles, structured spaces, MORK/PathMap integration, ACT artifacts, HE conformance machinery, and a developing split between local search, indexed shared-space execution, and type/elaboration machinery [1].

Second, the MORK-native supercompiler design contributes query-planner first optimization, a tiered pipeline, rich effect tracking, monotone MM2 analysis, cost-benefit thresholds, knowledge-base specialization, semi-naive or incremental maintenance, and AI-specific specialization paths [3].

Third, the multi-geometry IR design argues that direct MeTTa-to-MM2 lowering is too representation-committed. A better IR keeps multiple presentations visible: factor, DAG, trie, tensor, MM2, native C, and residual MeTTa [2].

Fourth, the CettaPack layer has to handle operational design constraints that are easy to miss in a source-only compiler: cross-space guards, structural hashes for meta-rule invalidation, incremental patching, temporal stability of profiles, a richer effect algebra, concrete bag-multiplicity protocol, PeTTa interop as an intermediate search backend, and strict benchmark discipline.

3 Architecture at a Glance

The proposed pipeline is:

```
MeTTa source + profile + workload stats + space schemas
        |
```

```
        v
Core normalization, meta-expansion, and effect analysis
        |
        v
MG-IR region graph
  native C | PathMap/trie | factor | tensor | MM2 | residual MeTTa
        |
        v
Planner chooses a mixed presentation and guard policy
        |
        v
Supercompiler specializes within the chosen presentation
        |
        v
Certificate skeleton and optional Lean/LeaTTa checking
        |
        v
CettaPack
  manifest + native kernel + PathMap/MORK artifacts
  + MM2 fragments + residual fallback + tests + certs
        |
        v
CeTTa runtime
  valid -> run
  patchable -> update then run
  replan -> fallback now and rebuild
  invalid -> fallback
```

This division is intentionally asymmetric. The expensive intelligence is in the planner, the effect system, the IR, and the artifact guards. The runtime hot path should be simple: check a compact guard vector, call a kernel if safe, otherwise fall back or patch.

4 Why CeTTa Adds Value

4.1 CeTTa is a host, not just an implementation language

CeTTa's value is not merely that it is written in C. A C compiler could emit native code without CeTTa. The value is that CeTTa is the runtime where MeTTa profiles, spaces, matchers, PathMap/MORK interfaces, structured spaces, and fallback semantics meet. A generated C kernel can borrow CeTTa runtime state rather than building its own parallel world.

A CettaPack kernel should be able to:

- receive a CeTTa runtime context;
- inspect profile and language-definition digests;
- access symbol IDs and interned atoms;

- scan PathMap or MORK-backed candidate sets;
- call exact native seeded rematch;
- allocate transient result atoms through arenas;
- stream answers through a visitor interface;
- read multiplicity records from the source space;
- fall back to generic CeTTa evaluation.

That collection of capabilities is the execution substrate for guarded supercompilation. PeTTa and MeTTaIL have important roles, but they do not replace this host boundary.

4.2 CeTTa has explicit semantic profiles

A supercompiler must know which semantics it is specializing. CeTTa's profile family makes that visible. A CettaPack should include a profile digest. It should not merely say "MeTTa". It should say whether the region was compiled under a compatibility profile, a principled profile, an extension profile, or a dependent-telescope profile. The numeric model, error contours, arity behavior, snapshot features, and extension surfaces are part of the artifact identity.

This is one of the strongest arguments for CeTTa as the host. If a runtime treats semantic differences as informal convention, the supercompiler has no clean guard key. If the runtime makes profiles and language-definition packs explicit, a CettaPack can fail closed when the semantic product changes.

4.3 CeTTa has the conservative PathMap boundary

The safe boundary is:

```
PathMap or MORK: choose candidates quickly.
CeTTa native matcher: decide exact bindings.
Kernel result builder: construct answers and multiplicities.
```

This is not a compromise; it is an optimization principle. Indexes are excellent at reducing search. They are dangerous if treated as the full semantics of matching. The generated kernel can get dramatic speedups by avoiding generic candidate enumeration while still preserving exact MeTTa semantics through native rematch.

4.4 CeTTa exposes mutation and snapshot surfaces

The hard cases in MeTTa are often about nondeterminism and mutable spaces. A kernel that reorders side effects may be wrong even if it is faster. A kernel that reads from a changing space may be wrong unless it has snapshot isolation, a lock, or an append-only epoch contract. CeTTa already has the conceptual machinery needed here: structured spaces, sequential branch behavior, snapshot surfaces, and effect-aware runtime boundaries.

A guarded supercompiler should not erase these details. It should use them to decide where each region belongs. Pure reads can become indexed C kernels. Append-only monotone regions may become MM2. Snapshot regions may become local episodes. Destructive mutation may remain residual or require a barrier.

5 Why Not Just PeTTa?

PeTTa is a strong Prolog-lineage runtime and should be treated with respect. It is particularly good at branch-heavy proof search. The CeTTa paper's benchmark ladder makes the split clear: CeTTa wins or is competitive on several C/runtime/data-heavy inference rows, while PeTTa wins decisively on pure branching search workloads [1].

The correct design response is not to choose one runtime and ignore the other. It is to use each where it belongs.

PeTTa should remain:

- a baseline for search performance;
- a semantic comparison point on shared fragments;
- a possible delegated backend for pure search episodes;
- a guide for designing CeTTa's local search machine.

CeTTa should remain the supercompilation host because it can integrate search with C kernels, MORK/PathMap, MM2, profiles, structured spaces, snapshots, ACT artifacts, and runtime fallback. A local PeTTa delegate inside a CettaPack is plausible. A pure-PeTTa artifact host for all mixed geometry execution is less natural.

The practical target is not "CeTTa beats PeTTa everywhere". The target is:

> Use PeTTa's strengths as a benchmark and, where useful, as a delegated backend, while making CeTTa strong enough on local search episodes that search-heavy regions no longer fail for avoidable abstract-machine reasons.

6 Why Not Just MeTTaIL?

MeTTaIL, or MeTTa-IL in the broader sense, is best understood here as a contract-first, proof-aware compilation layer. That is valuable. It can help define source contracts, normalize core regions, express proof obligations, or compile stable libraries. But the proposed supercompiler is not only a source-to-source or source-to-IL compiler. It is a live artifact system.

6.1 The live resources matter

A CettaPack depends on live resources:

- space identities and roles;
- space schemas and schema hashes;
- revision or epoch vectors;
- PathMap coverage and index artifacts;
- multiplicity records;
- selectivity statistics and drift thresholds;
- active profile and language-definition pack digests;
- backend availability and versions.

These are not just source terms. They are runtime conditions. A host must check them at the call site. If MeTTaIL were made responsible for all execution, it would need to become a runtime host for these same conditions. The simpler split is to let MeTTaIL describe and verify what it is good at, and let CeTTa guard and execute the live artifact.

6.2 Mixed artifacts are not a single IL

A production CettaPack may contain:

```
generated C shared object
PathMap or ACT index artifact
MM2 rule fragment
residual MeTTa fallback
certificate skeleton or Lean-checked certificate
profile and langdef digests
stats and schema dependency vectors
```

One could wrap all this in an IL abstraction, but the execution still happens in several backends. The design should acknowledge that heterogeneity rather than pretending every optimized region is really one IL program. The MG-IR gives a typed planning layer for the heterogeneity. CeTTa gives a runtime host for it.

6.3 Replicating CeTTa inside MeTTaIL would add a costly layer

The most direct way for MeTTaIL to take over CeTTa's role would be to represent every optimized artifact as an IL-level execution product. For stable libraries and source-level contracts that is a good idea. For hot runtime kernels over live spaces it is less efficient. The kernel wants to run where the data already live: next to CeTTa's symbol table, arena allocator, space records, PathMap and MORK handles, ACT-loaded artifact state, and exact matcher. Moving that path through a higher-level IL execution boundary introduces one or more costs: reifying native atoms as IL values, copying or serializing handles, allocating intermediate objects, dispatching through a general interpreter or VM layer, and then marshaling answers back into the runtime that owns the spaces.

One can avoid those costs only by giving MeTTaIL a native artifact runtime with the same low-level responsibilities: compact guard checks, per-space epoch vectors, schema and profile digests, PathMap iterators, MORK artifact handles, arena-safe result construction, native rematch, visitor-based streaming, patchable delta updates, and deoptimization to a reference evaluator. That would not eliminate CeTTa's role; it would rebuild a CeTTa-like role inside MeTTaIL. The cleaner architecture is to let MeTTaIL supply contracts and proof-aware transformations while CeTTa supplies the low-level, in-process execution host.

6.4 Exact rematch is a runtime seam

The highest-return first pass is PathMap candidate narrowing plus native seeded rematch. That depends on a concrete runtime seam: the generated kernel scans candidate atoms and calls the exact matcher. This seam is small enough to specify and test. If the whole execution is moved into MeTTaIL, one must either re-create the seam inside the IL runtime or trust a different matcher. That would not simplify the first killer pass.

6.5 Deoptimization and patching are host responsibilities

The guard lifecycle is a runtime policy:

```
valid      -> call kernel
patchable  -> apply bounded delta update, then call kernel
replan     -> run generic CeTTa now, rebuild asynchronously
invalid    -> fall back immediately
```

This policy depends on live state. It belongs in the execution host. MeTTaIL can help express the assumptions and certificates, but CeTTa is where those assumptions are checked against actual spaces, schemas, profiles, and artifacts.

6.6 The recommended relationship

The recommended relationship is complementary:

Layer	Role
MeTTaIL	Contract-first compilation, source normalization, stable library compilation, proof-aware transformation descriptions.
LeaTTa/Lean	Semantic floor, independent checker, certificate hardening, proof of key backend contracts.
MG-IR	Planner-visible operational presentations and safe coercions.
CeTTa	Runtime host for mixed artifacts, guard checking, patching, execution, exact rematch, fallback.

Thus Greg Meredith's question should be answered directly: do not avoid MeTTaIL; use it where it belongs. But do not force every specialized execution product to run inside MeTTaIL when the

real artifact is a profile-guarded mix of C, PathMap, MM2, residual MeTTa, and live space state. If MeTTaIL were extended until it could host that artifact with comparable efficiency, it would have acquired the same low-level duties that make CeTTa valuable: resident native data, exact runtime matching, artifact guards, patch/deopt policy, and C-level control over the hot path.

7 The Multi-Geometry IR

The MG-IR exists because a single low-level representation hides too much. It keeps the following distinction explicit:

```
semantic object: what the computation means
presentation:    how this meaning is operationalized
```

A relation may be presented as a trie for prefix scans, a sparse tensor for semiring contraction, a factor graph for local belief propagation, or an MM2 rule set for monotone saturation. A rule family may admit proof search, factor demand/supply, or forward saturation depending on profile, effects, and workload.

Presentation	Best use
Native C	Exact control, tight arithmetic, local search episodes, direct result construction, arena-local allocation.
PathMap/trie	Prefix scans, candidate narrowing, trie joins, support counts, heavy hitters, indexed space access.
Factor graph	PLN demand/supply, active subgraph extraction, cached messages, local truth-value kernels.
Tensor	Semiring contractions, batched sparse or dense kernels, possible GPU lowerings.
MM2	Monotone forward execution, sink-free saturation, priority scheduling, persistent indices.
Residual MeTTa	Reflection, dynamic program generation, rare regions, unsupported effects, debugging.

The important architectural decision is late lowering. MM2 is valuable, but not always the first or only target. The planner should first understand the operator family, ports, effects, laws, cache keys, statistics, and exactness requirements. Only then should it choose whether to lower to MM2, C, PathMap, tensor kernels, or residual code.

8 CettaPack v0.2

8.1 Artifact shape

A CettaPack is a directory or bundle with a manifest and one or more execution artifacts:

```
foo.cpack/
```

```
  manifest.json
  native/
    libfoo_kernel.so
  pathmap_artifacts/
    kb.act
    query_index.pm
  mm2/
    hot_region.mm2
  residual/
    fallback.metta
  certs/
    transform.cert
    lean_check.log
  tests/
    differential_cases.json
```

The manifest is the contract. The code is not admissible without it.

8.2 Dependency vector

The manifest must describe the identity of the compiled region:

```
identity:
  source_span_hash
  normalized_region_hash
  meta_expanded_region_hash
  match_pattern_hash
  langdef_pack_digest
  profile_digest
  operator_schema_hashes
  effect_policy_digest
  exactness_signature
```

It must also describe the spaces and artifacts:

```
spaces:
  - role
    space_id
    schema_hash
    revision_policy
    revision_or_epoch
    isolation_mode
    stats_signature
    multiplicity_source
    pathmap_coverage_hash
artifacts:
```

```
native_so_hash
pathmap_artifact_hash
mm2_fragment_hash
residual_fallback_hash
certificate_hash
```

CettaPack identity is not a single source hash and space revision. It is a dependency graph over code shape, profiles, effects, spaces, statistics, and artifacts.

8.3 Meta-rule invalidation

Meta-level rules can rewrite rules or change the effective structure of a region. Therefore the manifest must include the structural hash of the expanded region and the match pattern. If meta-expansion changes the shape, the old kernel is not the same kernel anymore. The loader must fall back or recompile even if the data spaces have not changed.

This rule should be fail-closed. A missing structural hash should mean "do not load", not "assume okay".

8.4 Guard states

A CettaPack guard checker should return one of four states:

```
VALID
PATCHABLE
REPLAN
INVALID
```

The difference is operationally important. A five-percent append-only delta in a compatible space may be patchable. A statistics drift beyond threshold may require replanning but not semantic invalidation. A schema hash mismatch is invalid. A source structural hash mismatch is invalid. A profile mismatch is invalid unless an explicit cross-profile theorem is provided.

9 Effect Algebra and Isolation

The effect vocabulary should be rich enough to support real decisions:

```
Read(resource, scope)
Append(resource, scope)
Write(resource, scope)
Observe(resource)
Create(resource)
Delete(resource)
```

Scopes and isolation policies may include:

```
Immutable
SnapshotIsolated
AppendOnlyEpoch
LockedRead
MutationBarrier
ThreadLocal
Unknown
```

A planner can then make conservative choices:

Effect and isolation	Likely optimization strategy
Read over immutable space	Strong candidate for C kernel and static PathMap artifact.
Read over snapshot	Candidate for C kernel if snapshot token is included in guard.
Append-only monotone	Candidate for patchable indices, semi-naive maintenance, or MM2 lowering.
Read plus destructive write	Needs barrier, lock, explicit sequential semantics, or residual fallback.
Create or delete resource	Usually not a first-pass supercompilation target unless region is fully scoped.
Unknown effect	Do not specialize except behind a safe residual wrapper.

Effect precision is also a performance feature. If a region is known to be append-only under an epoch policy, the runtime can patch instead of invalidating. If it is merely labeled "mutation", all such nuance is lost.

10 Pattern 1: Hot Match to PathMap plus Native Rematch

This is the first implementation target.

Input:

```
(match &S P R)
```

Output:

```
kernel_match_site_N(ctx, demand, visitor)
```

The kernel's plan is:

```
1. Check profile, langdef, structural, schema, and space guards.
2. Pick the most selective grounded prefix of P.
3. Iterate candidate atom IDs from PathMap or MORK.
```

```
4. If P is a conjunction, use product zipper or triejoin.
5. For each candidate tuple, call native seeded rematch.
6. If rematch succeeds, instantiate R.
7. Multiply or compose multiplicities according to demand mode.
8. Stream answers to the visitor.
9. On guard failure or unsupported features, run fallback.metta.
```

10.1 Multiplicity protocol

PathMap should store canonical support, not result-bag semantics. The kernel reconstructs multiplicity from the source spaces:

```
single atom result:
  output count = source multiplicity

conjunction:
  output count = semiring/product composition of source counts
  after exact binding compatibility is confirmed

support-only demand:
  output count is ignored only when demand says support mode
```

This protocol must be explicit because duplicates are semantically real in MeTTa. Deduplication for speed is a semantic change unless the caller asked for support.

10.2 Correctness contract

The essential contract is:

```
PathMap candidate selector is sound.
Native rematch is exact on the accepted fragment.
Result template instantiation matches generic CeTTa.
Multiplicity is reconstructed from space records.
Fallback is semantically identical to generic evaluation.
```

The first Lean proof milestone should target this contract.

11 Pattern 2: Search Episodes, MAM, and PeTTa Interop

A hot backward chainer should not always be interpreted through generic MeTTa evaluation. It can be specialized into a local search episode.

The native episode has the shape:

```
registers for bound variables
compact choice frames
trail with watermarks
scratch arena mark/reset
fixed rule dispatch
PathMap-backed fact iterators
native exact rematch at boundaries
streaming answer visitor
```

This resembles a WAM or MAM-style machine, but it should remain hidden behind a generated kernel. It should not become new user-visible Prolog semantics. The source remains MeTTa. The artifact is an implementation strategy for a guarded hot region.

PeTTa interop is a useful intermediate. For a pure local search region that fits a checked fragment, the CettaPack may delegate to PeTTa and then import the result bag. The manifest must state the fragment contract, translation route, binding semantics, multiplicity policy, and fallback path. CeTTa remains responsible for deciding whether delegation is valid.

12 Pattern 3: PLN Factor Kernels

PLN often benefits from a geometry shift. Instead of interpreting a backward proof tree directly, the compiler can present the problem as:

```
backward demand over rule/factor graph
active subgraph extraction
forward truth-value supply
cached local messages
unboxed truth-value arithmetic
```

The human supplies rule schemas with ports and laws:

```
rule name
premise ports
conclusion ports
truth-value formula
monotonicity or approximation status
cache key fields
allowed lowerings
```

The compiler specializes local formulas, role-indexed lookups, cache keys, and active-subgraph traversal. Some arithmetic stays in C. Some monotone update loops may lower to MM2. Some batched kernels may later lower to sparse or dense tensor code.

This is an example where the MG-IR is essential. A raw MM2 lowering may not expose the factor semantics, cache boundaries, demand propagation, or truth-value formula structure.

13 Pattern 4: Monotone MM2 Lowering

MM2 is an excellent backend when the region is monotone and sink-free. The path should be late:

```
MeTTa region
  -> normalized effect-aware core
  -> MG-IR monotone presentation
  -> optional supercompiled/specialized presentation
  -> MM2 skeleton and priorities
  -> CeTTa/MORK step! execution
```

The planner should ask:

- Is the region append-only or otherwise monotone?
- Are there destructive sinks or hidden side effects?
- Is the schedule statically analyzable?
- Do priorities preserve the source ordering constraints?
- Can incremental pattern matching and persistent indices be reused?
- Does MM2 lowering beat native C or PathMap kernels for this region?

A region that fails these tests should not be forced into MM2. It may be better as native C, residual MeTTa, or a local episode.

14 Structural Hashes and Language Definition Packs

The source span alone does not identify a compiled region. The effective region is produced after parsing, imports, macro expansion, meta-rule application, profile selection, and operator schema resolution. A CettaPack should therefore store several hashes:

```
source_span_hash:
  the original text region or AST span

normalized_region_hash:
  the core region after ordinary normalization

meta_expanded_region_hash:
  the region after macro and meta-rule expansion

match_pattern_hash:
  the structural identity of the compiled pattern

langdef_pack_digest:
```

```
  the language rule pack and profile-specific semantics

operator_schema_hashes:
  the schemas used by the planner and kernel generator
```

The loader should compare all relevant hashes. If a meta-rule rewrite changes the pattern, the old kernel must not be reused. If a profile upgrade changes the schema under the same human-readable name, the old kernel must not be reused. If only a claim level is upgraded without changing semantic content, the semantic digest should remain stable; but that requires the digest rules to be explicit.

15 Incremental Maintenance and Temporal Stability

A binary guard model is too crude. Many useful spaces grow gradually. If a space is append-only and the new atoms fall within a bounded delta, the runtime should patch:

- PathMap entries;
- support counters;
- selectivity statistics;
- semi-naive delta tables;
- cached active-subgraph summaries;
- small materialized lookup tables.

The manifest should include thresholds:

```
max_append_delta
max_patch_time
max_stats_drift
max_index_coverage_loss
expected_validity_window
expected_remaining_calls
compile_cost_estimate
```

This lets the planner ask whether specialization will amortize. A site that is hot for one second and then cold should not receive a large compilation budget. A site that remains hot across runs deserves deeper specialization and perhaps Lean-checked admission.

Temporal stability is therefore part of profiling. The profiler should track not only how often a region runs, but how stable its code shape, space dependencies, selectivity, and effect environment are.

16 Certificates and Verification Phasing

The certificate story should progress in phases.

16.1 Phase 1: data certificates and differential testing

The first CettaPacks should emit structured certificates:

```
source hashes
profile and langdef digests
effect summary
space dependency vector
operator schemas
rewrite steps
backend schema name
fallback hash
test corpus hash
```

They should be tested differentially against generic CeTTa and, where applicable, PeTTa and HE. This is not a proof, but it is a disciplined start.

16.2 Phase 2: check selected transformations

Lean or LeaTTa should next check small, high-value claims:

- native seeded rematch parity;
- candidate-soundness of PathMap selectors;
- preservation of result-template instantiation;
- pure normalization and inlining steps;
- monotone append-only MM2 lowering for a small fragment;
- multiplicity reconstruction for simple conjunctions.

16.3 Phase 3: proof-carrying CettaPack admission

Only later should the loader require machine-checked certificates for selected trusted classes. This avoids blocking the MVP while creating a path toward real assurance.

The key discipline is honesty. A CettaPack should report whether it is bag-tested, rule-sound-on-fragment, or fully checked. It should not claim proof because it passed tests.

17 Benchmarking Discipline

Benchmarks should compare the artifact against multiple baselines:

- generic CeTTa;

- compiled CeTTa with guard checks included;
- PeTTa when the region is in a shared or translated fragment;
- HE when an HE-compatible path exists;
- raw MM2 or MORK-specific path when relevant;
- residual fallback.

Guard overhead must be measured. A useful benchmark matrix includes:

```
fast-path guard latency
cross-space guard vector latency
PathMap candidate count
native rematch count
answer construction time
multiplicity reconstruction time
fallback frequency
patch time for append-only deltas
statistics drift and replan frequency
compile-time cost
amortization break-even point
memory peak and answer materialization cost
```

Correctness benchmarks should include:

```
duplicate atoms
co-reference variables
distinct variables
empty result bags
multi-space joins
profile mismatch
schema migration
meta-rule structural change
append-only patching
destructive mutation invalidation
PathMap-disabled fallback
```

The rule is simple: never report only the generated kernel time. Include guard overhead and the fallback baseline.

18 Hyperseed and OmegaHive Framing

The Hyperseed reading is useful but should not be a hard dependency for the first implementation. In Hyperseed terms, a CettaPack is a context-bounded transformation. Its manifest is the

guarded contract. Its source hashes, profile digests, schema hashes, statistics, and artifact hashes are provenance. Its guard lifecycle is a resource selection policy.

This also maps naturally to an OmegaHive-style artifact ledger. Each CettaPack can be cached and shared by:

```
source hash
expanded region hash
profile digest
langdef digest
space schema vector
operator schema vector
backend family
exactness signature
```

A compilation worker can produce a CettaPack. A Lean worker can harden or check the certificate. A memory curator can deduplicate equivalent artifacts. Runtime agents can reuse a pack only if their dependency vectors match or can be patched safely.

19 Tensor and GPU Roadmap

Tensor lowering is important but should be late. It depends on the same foundations as the other paths: effect safety, space schema stability, operator schemas, and exact or approximate witness propagation.

A tensor path is appropriate when:

- a relation or factor region is finite and bounded;
- the semiring is known;
- a sparse or dense layout can be chosen;
- sharding and reattachment are well specified;
- the error budget is explicit if approximation is used;
- the kernel amortizes transfer and setup costs.

The roadmap should include a placeholder for tensor/GPU lowerings after match kernels, guard lifecycle, incremental maintenance, search episodes, and PLN factor kernels are stable.

20 Roadmap

A practical roadmap is:

Milestone 0: instrumentation

Add profiling for hot regions, temporal stability, space dependency vectors, effect summaries, candidate counts, result counts, and guard overhead.

Milestone 1: hot match kernel

Compile one fixed match site to C using PathMap candidate narrowing plus native seeded rematch. Preserve multiplicity. Test duplicates and co-reference. Provide fallback.

Milestone 2: CettaPack loader

Implement manifest parsing, dependency-vector guard checks, structural hash validation, cross-space guards, and the valid/patchable/replan/ invalid lifecycle.

Milestone 3: patchable append-only spaces

Support bounded updates to PathMap entries, support counters, and statistics. Add drift thresholds and replan triggers.

Milestone 4: certificate skeletons

Emit transform.cert as structured data. Differentially test every pack. Start Lean work on native seeded rematch parity.

Milestone 5: search episode

Either integrate a checked PeTTa delegate for pure search episodes or implement a native local search episode with choice frames, trail/watermark rollback, scratch arenas, and streamed answers.

Milestone 6: PLN factor kernel

Add operator schemas for selected PLN rules. Generate unboxed C truth-value formulas and active-subgraph kernels.

Milestone 7: monotone MM2 lowering

Lower sink-free append-only regions to MM2 only after effect proofs or strong tests establish safety.

Milestone 8: tensor and approximate kernels

Add tensor/GPU lowerings and approximate presentations with explicit error witnesses only after exact kernels are stable.

21 Conclusion

The proposed architecture is a synthesis:

> LeaTTa/Lean-checked, MG-IR-planned, CeTTa-hosted supercompilation into mixed C, PathMap/MORK, MM2, factor, tensor, PeTTa-delegated, and residual MeTTa artifacts.

The most important improvement over the initial design is that guards must cover both data state and code shape. A CettaPack depends on a normalized semantic region, a meta-expanded structural hash, a language-definition digest, operator schemas, profile, effects, space vectors, artifact hashes, and statistics. This is the price of making aggressive optimization safe in a reflective language.

The answer to "why CeTTa?" is now sharper. CeTTa is not merely a fast C implementation. It is the host where mixed optimized artifacts can be run, patched, guarded, deoptimized, and checked against exact MeTTa semantics. PeTTa remains the search engine to learn from and sometimes call. MeTTaIL remains a contract-first layer to use. MM2 remains a powerful monotone backend. MORK and PathMap remain the indexed substrate. CeTTa is the runtime junction where those pieces can become one guarded system.

A Appendix for Coding Agents

This appendix gives concrete guidance for implementation agents. It is less philosophical and more operational.

A.1 Prime directive

Do not replace the evaluator. Build guarded, site-specific, semantics-preserving specialization paths. Every generated kernel must have a manifest guard and a fallback path. Unsupported cases should refuse compilation or deopt, not guess.

A.2 CettaPack manifest v0.2 sketch

```
{
  "artifact_kind": "cettapack",
  "manifest_version": "0.2",

  "identity": {
    "source_span_hash": "...",
    "normalized_region_hash": "...",
    "meta_expanded_region_hash": "...",
    "match_pattern_hash": "...",
    "langdef_pack_digest": "...",
    "profile_digest": "...",
    "operator_schema_hashes": ["..."],
    "effect_policy_digest": "...",
```

```
  "exactness_signature": "exact"
},

"effects": {
  "classes": [
    "Read(space:evidence, scope:snapshot)",
    "Append(space:derived, scope:epoch)"
  ],
  "isolation": "SnapshotIsolated"
},

"spaces": [
  {
    "role": "evidence",
    "space_id": "...",
    "schema_hash": "...",
    "revision_policy": "append_only_epoch",
    "revision_or_epoch": "...",
    "stats_signature": "...",
    "multiplicity_source": "space_records",
    "pathmap_coverage_hash": "..."
  },
  {
    "role": "rules",
    "space_id": "...",
    "schema_hash": "...",
    "revision_policy": "immutable",
    "revision_or_epoch": "...",
    "stats_signature": "..."
  }
],

"guards": {
  "fast_path": "dependency_vector_equal",
  "patch_path": "append_only_delta_under_budget",
  "replan_path": "stats_drift_over_threshold",
  "invalid_path": "fallback_to_cetta"
},

"artifacts": {
  "native_so_hash": "...",
  "pathmap_artifact_hashes": ["..."],
  "mm2_fragment_hashes": ["..."],
  "residual_metta_hash": "...",
  "transform_cert_hash": "..."
},

"amortization": {
```

```
    "compile_cost_estimate_ms": 0,
    "expected_calls": 0,
    "expected_validity_window_ms": 0,
    "max_patch_cost_ms": 0,
    "max_stats_drift": 0.0,
    "max_append_delta_atoms": 0
  }
}
```

A.3 Guard check pseudocode

```
function check_cettapack_guard(pack, runtime):
    if pack.identity.profile_digest != runtime.profile_digest:
        return INVALID

    if pack.identity.langdef_pack_digest != runtime.langdef_digest:
        return INVALID

    if current_meta_expanded_hash(pack.source_span) !=
       pack.identity.meta_expanded_region_hash:
        return INVALID

    if current_match_pattern_hash(pack.source_span) !=
       pack.identity.match_pattern_hash:
        return INVALID

    for dep in pack.spaces:
        live = runtime.lookup_space(dep.space_id)

        if live.schema_hash != dep.schema_hash:
            return INVALID

        state = check_revision_policy(dep, live)

        if state == INVALID:
            return INVALID

        if state == PATCHABLE:
            if not patch_budget_allows(dep, live):
                return REPLAN
            mark_patch_needed(dep)

        if stats_drift(dep.stats_signature, live.stats) >
           pack.amortization.max_stats_drift:
            return REPLAN

    if any_patch_needed():
```

```
        return PATCHABLE

    return VALID
```

A.4 Patch lifecycle pseudocode

```
function run_cettapack(pack, ctx, visitor):
    state = check_cettapack_guard(pack, ctx.runtime)

    if state == VALID:
        return call_native_kernel(pack, ctx, visitor)

    if state == PATCHABLE:
        ok = apply_patches(pack, ctx.runtime)
        if ok and check_cettapack_guard(pack, ctx.runtime) == VALID:
            return call_native_kernel(pack, ctx, visitor)
        else:
            schedule_replan(pack)
            return run_fallback(pack, ctx, visitor)

    if state == REPLAN:
        schedule_replan(pack)
        return run_fallback(pack, ctx, visitor)

    if state == INVALID:
        return run_fallback(pack, ctx, visitor)
```

A.5 Structural hash computation

A structural hash should be computed after all transformations that change the effective region:

```
parse source span
resolve imports visible to the span
apply macro expansion
apply meta-rule expansion
normalize alpha-equivalent binders
normalize variable IDs or de Bruijn indices
record operator schema IDs
record profile-specific special-form interpretations
serialize canonical tree
hash canonical serialization
```

Do not include non-semantic evidence levels in this hash. Do include semantic rule content, operator schemas, and profile-relevant behavior.

A.6 MVP match kernel interface

Suggested C shape:

```
typedef enum {
    CETTA_VISIT_CONTINUE,
    CETTA_VISIT_STOP,
    CETTA_VISIT_ERROR
} CettaVisitCtl;

typedef struct CettaKernelCtx CettaKernelCtx;
typedef struct CettaAtom CettaAtom;
typedef struct CettaVisitor CettaVisitor;

typedef CettaVisitCtl (*CettaAnswerFn)(
    CettaKernelCtx *ctx,
    const CettaAtom *answer,
    size_t multiplicity,
    void *user_data);

int cetta_kernel_match_site_N(
    CettaKernelCtx *ctx,
    CettaAnswerFn visitor,
    void *user_data);
```

The kernel borrows runtime state. It must not own global state. It must not mutate input spaces unless the manifest explicitly says so.

A.7 MVP match kernel algorithm

```
function kernel_match_site_N(ctx, visitor, user_data):
    guard = check_guard(ctx)
    if guard != VALID:
        return run_fallback(ctx, visitor, user_data)

    iterator = pathmap_prefix_iterator(ctx.space, compiled_prefix)

    for candidate in iterator:
        bindings = native_seeded_rematch(compiled_pattern, candidate)

        if bindings.success:
            answer = instantiate_result_template(bindings)
            mult = read_multiplicity(candidate)
            ctl = visitor(ctx, answer, mult, user_data)

            if ctl == STOP:
                return OK
```

```
            if ctl == ERROR:
                return ERROR

    return OK
```

For conjunctions, candidate is a tuple and multiplicity is composed after exact binding compatibility is confirmed.

A.8 Certificate skeleton

```
(supercompile-cert
  :manifest-version 0.2
  :source-span-hash Hs
  :normalized-region-hash Hn
  :meta-expanded-region-hash Hm
  :match-pattern-hash Hp
  :profile-digest Hprof
  :langdef-pack-digest Hlang
  :effect-summary E
  :space-dependency-vector V
  :backend-schema pathmap-candidate-native-rematch-v1
  :exactness exact
  :steps
    ((parse-and-normalize step-id-1)
     (meta-expand step-id-2)
     (choose-pathmap-prefix step-id-3)
     (prove-or-claim-candidate-soundness step-id-4)
     (native-seeded-rematch-parity step-id-5)
     (result-template-instantiation step-id-6)
     (multiplicity-reconstruction step-id-7)
     (residualize-to-c-kernel step-id-8)))
```

Claim levels should be explicit:

```
bag-tested-adequate
rule-sound-on-fragment
rule-sound
unchecked-experimental
```

A.9 Differential tests for match kernels

Every MVP kernel needs tests for:

1. one ground match;
2. no-result match;

3. multiple result bag;
4. duplicate atoms and multiplicity;
5. repeated variable co-reference;
6. distinct variables that must not collapse;
7. result-template construction;
8. cross-space join if supported;
9. profile mismatch fallback;
10. schema hash mismatch fallback;
11. meta-expanded structural hash mismatch fallback;
12. append-only patching path;
13. destructive write invalidation;
14. PathMap disabled fallback;
15. comparison with generic CeTTa;
16. comparison with PeTTa or HE when the fragment allows it.

A.10 Anti-patterns

Avoid these mistakes:

- Do not treat PathMap keys as full MeTTa bindings.
- Do not deduplicate result bags unless demand mode says support.
- Do not omit structural hashes because space revisions look stable.
- Do not assume a schema name implies schema identity.
- Do not assume profile upgrades preserve kernel validity.
- Do not compile destructive shared-space mutation to MM2.
- Do not hide approximate behavior behind exact function names.
- Do not call a generated kernel when guard status is unknown.
- Do not make Lean checking a milestone-1 blocker.
- Do not benchmark without guard overhead.
- Do not remove the residual fallback.

A.11 Definition of done for milestone 1

Milestone 1 is complete when:

1. one nontrivial fixed match site compiles to C;
2. generated C uses PathMap only for candidate selection;
3. native seeded rematch is still called;
4. multiplicity is reconstructed from source space records;
5. duplicates and co-reference are tested;
6. manifest includes profile, langdef, structural, schema, and space fields;
7. guard failure forces fallback;
8. fallback returns the same result bag as generic CeTTa;
9. a certificate skeleton records the transformation;
10. benchmark includes generic CeTTa, compiled CeTTa with guards, PeTTa where applicable, and HE where applicable.

References

[1] Z. Goertzel and Oruzi. *CeTTa: A Direct C Runtime for MeTTa with Verified Translation*. Working paper, July 2026.

[2] B. Goertzel. *Supercompiling MeTTa to Mixed MeTTa+MM2 via a Multi-Geometry IR*. Working design note, April 2026.

[3] B. Goertzel. *A Comprehensive MORK-Native Supercompiler for MeTTa+MM2: Core Design and AI Specializations*. Working design note, October 2025.

[4] Z. Goertzel and Oruzi. *MeTTaIL: Contract-First Boot Compilation for Direct C MeTTa Runtimes*. Working paper, 2026.